Established – 1961 Subject: OS DBMS

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE

ULHASNAGAR – 421 003

CERTIFICATE

This is to certify that Mr. Subham.N.Mahanty of S.Y. Information Technology (SYIT) Roll No. 2542022 has satisfactorily completed the Open Source DataBase Management System Mini Project entitled Income Tax Record Management Database During the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my respected project guide and subject teacher for providing me with the valuable opportunity to work on the project titled “Income Tax Record Management System”. This project has helped me in gaining practical knowledge about database management systems and the implementation of taxation logic using SQL and GUI-based application development.

I am extremely thankful to my guide for their continuous support, valuable suggestions, and guidance throughout the development of this project. Their encouragement and timely feedback helped me to understand the concept of advanced tax calculation methods such as advance tax payment, tax adjustment, and carry forward of excess tax to the next financial year.

I would also like to thank the Head of the Department and all the faculty members of the Computer Science Department of my college for their cooperation and support during the completion of this project. This project would not have been possible without their help and motivation.

I am grateful to my friends and classmates for their constant encouragement and support during the project development phase. Finally, I would like to thank my parents for their moral support and motivation which helped me in completing this project successfully.

This project has enhanced my understanding of real-world taxation systems and database implementation through MySQL Workbench and GUI-based project deployment.

ABSTRACT

The Income Tax Record Management System is a database-driven project developed to manage taxpayer information and perform tax-related calculations efficiently using MySQL Workbench and a GUI-based interface. The main objective of this project is to store and manage details of taxpayers such as personal information, income details, advance tax payments, and final tax payments in a structured and organized manner.

This system is designed to automate the process of maintaining taxation records by implementing various database tables such as Taxpayer, Income, Tax Slab, Advance Tax, Tax Calculation, Tax Adjustment, and Tax Payment. The project helps in managing financial year-wise income records and advance tax installments paid by taxpayers. Based on the stored income details, the system performs tax computation and determines the final payable tax amount.

The system also maintains records of advance tax payments made during a financial year and calculates the difference between total tax liability and the advance tax paid. In cases where excess tax is paid, the remaining amount is recorded and carried forward for adjustment in the next financial year. This helps in maintaining transparency and accuracy in taxation records.

The project is implemented using MySQL as the backend database and SQL Workbench for executing queries, while the graphical user interface (GUI) provides a user-friendly environment for data entry and result display. This system reduces manual effort, minimizes calculation errors, and ensures proper management of income tax records.

Overall, the Income Tax Record Management System provides an efficient and reliable platform for managing taxpayer data and taxation records in a systematic manner.

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	5-6
2	Problem Definition	7-8
3	Objectives of the Project	9
4	Scope of the Project	10-11
5	Requirement Specification	12-13
6	System Design	14-16
7	Database Design	17-20
8	UML Diagrams	21-23
9	SQL Implementation	24-28
10	System Testing and Result	29-32
11	Security, Backup and Recovery	33
12	Future Scope and Conclusion	34
13	References	35
14	Glossary	36

1. INTRODUCTION

A database system is a structured method of storing, organizing, and managing large volumes of data efficiently. It allows information to be stored in related tables so that data can be accessed, updated, and retrieved accurately. Database systems reduce redundancy, prevent inconsistencies, and ensure secure handling of sensitive information. In government departments such as the Income Tax Department, database systems play a crucial role in managing financial records securely and efficiently.

The Income Tax Record Management System with Advance Tax Adjustment is a database-driven system designed to manage taxpayer details, income records, tax calculation, advance tax payments, and excess tax adjustments. In traditional manual systems, tax records are maintained using physical registers or spreadsheets, which often result in errors, duplicate entries, incorrect calculations, and difficulty in tracking previous year adjustments.

In real-world taxation systems, taxpayers may pay advance tax or may have excess tax deducted through TDS (Tax Deducted at Source). If the deducted tax amount exceeds the calculated tax liability, the excess amount is either refunded or adjusted in the next financial year. Managing such adjustments manually becomes complicated and error-prone.

This project implements an advanced taxation mechanism where:

- Tax is calculated based on income slabs
- Advance tax payments are recorded
- Excess tax is automatically calculated
- Carry-forward adjustment is applied to the next financial year
- Final payable tax is computed after adjustment

The system uses MySQL as an open-source relational database management system. Tables are structured to maintain relationships between taxpayers, income records, tax slabs, tax calculations, advance payments, and adjustment records. Database constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK ensure data integrity and consistency.

Transaction management is implemented to ensure that tax calculation, payment entry, and adjustment processing occur safely. If any step fails, the transaction is rolled back to prevent partial updates.

In conclusion, this project demonstrates how a relational database can be used to manage income tax records efficiently with real-world advance tax adjustment logic. It provides practical exposure to database design, SQL queries, constraints, joins, views, and transaction control mechanisms.

2. PROBLEM DEFINITION

In traditional income tax record management systems, tax calculations and payment records are often maintained manually or using simple spreadsheet tools. While basic tax calculation may be manageable, handling advance tax payments, excess deductions, and carry-forward adjustments becomes highly complex without a structured database system.

In real-world taxation, taxpayers often pay advance tax in installments or have tax deducted at source (TDS). Sometimes, the total tax paid exceeds the actual tax liability. In such cases, the excess amount must either be refunded or adjusted in the next financial year. Managing this adjustment process manually leads to several serious problems.

The absence of an automated database-driven advance tax system creates the following issues:

• **Incorrect Tax Adjustment:**
Excess tax paid in one financial year may not be properly adjusted in the next year, leading to wrong tax liability.

• **Duplicate Tax Records:**
Without proper validation, multiple payment entries for the same taxpayer and financial year may cause confusion.

• **Data Redundancy:**
Taxpayer and payment information may be stored repeatedly in different records.

• **Calculation Errors:**
Manual adjustment of advance tax and carry-forward credit increases chances of arithmetic mistakes.

• **Inconsistent Records:**
Tax calculation and payment records may not match if updates are made partially.

• **Difficulty in Tracking Advance Payments:**
Maintaining installment-wise advance tax records manually is time-consuming.

• **No Proper Refund Tracking:**
Excess tax may not be accurately identified or processed for refund.

• **Lack of Data Integrity:**
Payment records may exist without proper linkage to tax calculation records.

• **Security Risks:**
Sensitive taxpayer information such as PAN and income details may not be protected.

• **Scalability Issues:**
As the number of taxpayers increases, managing advance tax adjustments manually becomes inefficient.

These challenges highlight the need for a structured relational database system that not only calculates tax based on slabs but also:

- Records advance tax payments
- Identifies excess tax
- Automatically carries forward excess amount
- Adjusts tax liability in the next financial year
- Maintains secure and consistent records

The proposed Income Tax Record Management System with Advance Tax Adjustment solves these issues by implementing relational database design, constraints, and transaction management using MySQL.

3. OBJECTIVES OF THE PROJECT

The main objectives of the Income Tax Record Management System with Advance Tax Adjustment are as follows:

• To design and develop a structured relational database system for managing income tax records using MySQL.

• To store and manage taxpayer details such as name, PAN number, address, contact information, and financial year records efficiently.

• To record income details of taxpayers for different financial years in a structured format.

• To calculate tax automatically based on predefined income tax slabs.

• To implement an advance tax mechanism where taxpayers can pay tax in installments before final assessment.

• To store advance tax and TDS payment records separately for accurate tracking.

• To identify excess tax paid after comparing calculated tax liability with total tax paid.

• To implement carry-forward logic where excess tax from the previous financial year is adjusted in the next financial year.

• To calculate final payable tax after deducting advance tax and previous year adjustments.

• To apply database constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK to ensure data integrity and prevent duplicate records.

• To implement transaction management for safe processing of tax calculation, payment recording, and adjustment operations.

• To generate financial year–wise tax reports and adjustment summaries using SQL queries, JOIN operations, GROUP BY, and HAVING clauses.

• To reduce manual errors and improve efficiency in managing complex tax adjustment scenarios.

4. SCOPE OF THE PROJECT

The Income Tax Record Management System with Advance Tax Adjustment is designed to manage taxpayer records, income details, tax calculations, advance tax payments, excess tax identification, and carry-forward adjustments using a structured relational database approach.

Unlike basic tax management systems that only calculate tax based on income slabs, this system incorporates real-world taxation concepts such as advance tax installments, excess payment detection, and automatic adjustment of previous year credits.

Users of the System

• **Administrator:**
Responsible for managing tax slabs, monitoring system operations, maintaining database security, and controlling user privileges.

• **Tax Officer / Staff:**
Handles taxpayer registration, income entry, advance tax recording, final tax calculation, and adjustment processing.

• **Tax Consultants:**
Can use the system to manage multiple client tax records efficiently.

• **Students (Academic Use):**
Helps in understanding advanced taxation logic integrated with database systems.

Areas of Use

• **Educational Purpose:**
The system demonstrates real-world tax processing concepts such as advance tax adjustment and carry-forward mechanism using DBMS concepts.

• **Small Tax Consultancy Offices:**
Can use the system to maintain structured records of taxpayer liabilities and adjustments.

• **Administrative Record Management:**
Allows structured storage of financial year–wise income, tax liability, payment, and adjustment data.

Functional Scope

The system supports:

Taxpayer registration and PAN validation

- Financial year–wise income recording
- Slab-based tax calculation
- Advance tax installment entry
- Comparison of calculated tax vs. total tax paid
- Automatic identification of excess or shortfall
- Carry-forward of excess tax to next financial year
- Final payable tax computation
- Payment status tracking (PAID / PENDING / REFUND)
- Report generation using SQL queries

Academic Limitations

• The system is developed primarily for academic and demonstration purposes.

• It does not integrate with actual government tax portals.

• Real-time online filing and bank API integration are not included.

• Complex national tax rules and exemptions are simplified for educational clarity.

Overall, the scope of the project is to demonstrate how a relational database system can handle advanced income tax processing, including advance payments and carry-forward adjustments, while maintaining data integrity, security, and consistency.

5. REQUIREMENT SPECIFICATION

5.1 Hardware Requirements

The following hardware requirements are necessary to run the Income Tax Record Management System with Advance Tax Adjustment:

• **Computer / Laptop:**
A standard desktop or laptop capable of running MySQL Server and MySQL Workbench.

• **Minimum RAM:**
At least 4 GB RAM for smooth execution of database operations and transaction processing.

• **Storage:**
Minimum 10 GB free disk space to store database files, software tools, and backup files.

• **Processor:**
Intel i3 or higher (or equivalent) for efficient query execution and tax calculation processing.

5.2 Software Requirements

The following software tools are required for development and execution of the system:

Software	Purpose
MySQL Server	Used to create, store, and manage income tax database
MySQL Workbench	Provides graphical interface to write and execute SQL queries
SQL	Query language used for database operations and tax logic implementation
Operating System (Windows / Linux)	Platform required to install and run MySQL tools
mysqldump Utility	Used for database backup and recovery
Web Browser (Optional)	Access documentation and learning resources

The above configuration ensures that the system can efficiently handle:

- Slab-based tax calculation
- Advance tax installment processing
- Carry-forward adjustment logic
- Transaction management
- Secure data storage

6. SYSTEM DESIGN

6.1 System Architecture (Conceptual)

The Income Tax Record Management System with Advance Tax Adjustment follows a database-centered architecture where users interact with the system through SQL queries executed on a MySQL database server.

The architecture ensures secure data storage, structured tax calculation, advance payment tracking, and automatic adjustment of excess tax in the next financial year.

In this system:

• The user (Administrator / Tax Officer) interacts through MySQL Workbench.
• SQL queries are executed to insert income records, calculate tax, record advance payments, and process adjustments.
• The MySQL Server processes queries, validates constraints, and manages transactions.
• All tax-related data is stored in structured relational tables.

Core Components of Architecture

User Layer
Admin / Tax Officer enters taxpayer details, income records, advance tax payments, and initiates tax calculation.

Application / Query Layer
SQL commands are executed through MySQL Workbench.

Database Server Layer (MySQL Server)
Handles:

- Data storage
- Slab matching
- Advance tax comparison
- Excess identification
- Carry-forward adjustment

- Transaction control

Database Tables

The following tables store system data:

• Taxpayer
• Income
• Tax_Slab
• Tax_Calculation
• Advance_Tax
• Tax_Adjustment
• Tax_Payment

Conceptual Architecture Diagram

6.2 Advance Tax Processing Flow

The system follows the below logical flow:

1. Taxpayer registration
2. Income entry for financial year
3. Tax slab identification
4. Tax calculation
5. Advance tax installment recording
6. Compare calculated tax with total advance tax paid
7. If advance tax > calculated tax → excess detected
8. Excess stored in Tax_Adjustment table
9. In next financial year, excess is deducted from new tax liability
10. Final payable tax calculated

Advance Tax Adjustment Flow Diagram

6.3 Transaction Management in System Design

The system uses transaction control to ensure safe processing:

• START TRANSACTION
• COMMIT
• ROLLBACK

For example, during tax finalization:

- Tax calculation is inserted
- Advance tax is summed
- Adjustment is computed
- Final payable tax is stored

If any step fails, the entire process is rolled back to maintain data consistency.

Overall, this system architecture ensures:

- Structured relational design
- Secure data handling
- Accurate advance tax processing
- Automatic carry-forward adjustment
- Reliable transaction control

7. DATABASE DESIGN

The Income Tax Record Management System is designed using a relational database model. All entities are connected using primary key and foreign key relationships to maintain referential integrity.

The database consists of the following major tables:

1. Taxpayer
2. Income
3. Tax_Slab
4. Tax_Calculation
5. Advance_Tax
6. Tax_Adjustment
7. Tax_Payment

7.1 Taxpayer Table

Stores basic details of registered taxpayers.

Field Name	Data Type	Description
taxpayer_id	INT (PK)	Unique ID of taxpayer
name	VARCHAR(100)	Name of taxpayer
pan_number	VARCHAR(15)	PAN number (Unique)
date_of_birth	DATE	Date of birth
contact_number	VARCHAR(15)	Mobile number
email	VARCHAR(100)	Email address
address	VARCHAR(200)	Residential address

Primary Key: taxpayer_id

7.2 Income Table

Stores income details financial year–wise.

Field Name	Data Type	Description
income_id	INT (PK)	Unique income record ID
taxpayer_id	INT (FK)	References Taxpayer
financial_year	VARCHAR(10)	Example: 2025-26
salary_income	DECIMAL(10,2)	Income from salary
business_income	DECIMAL(10,2)	Business income
other_income	DECIMAL(10,2)	Other income
total_income	DECIMAL(10,2)	Total calculated income

Foreign Key: taxpayer_id → Taxpayer(taxpayer_id)

7.3 Tax_Slab Table

Stores slab-based tax percentage.

Field Name	Data Type	Description
slab_id	INT (PK)	Slab ID
min_income	DECIMAL(10,2)	Minimum income range
max_income	DECIMAL(10,2)	Maximum income range
tax_percentage	DECIMAL(5,2)	Applicable tax rate

Primary Key: slab_id

7.4 Tax_Calculation Table

Stores calculated tax before adjustment.

Field Name	Data Type	Description
calculation_id	INT (PK)	Unique calculation ID
taxpayer_id	INT (FK)	References Taxpayer

financial_year	VARCHAR(10)	Financial year
total_income	DECIMAL(10,2)	Total income
calculated_tax	DECIMAL(10,2)	Tax as per slab

Foreign Key: taxpayer_id → Taxpayer

7.5 Advance_Tax Table

Stores advance tax installment payments.

Field Name	Data Type	Description
advance_id	INT (PK)	Unique advance payment ID
taxpayer_id	INT (FK)	References Taxpayer
financial_year	VARCHAR(10)	Financial year
installment_no	INT	Installment number (1–4)
amount_paid	DECIMAL(10,2)	Amount paid
payment_date	DATE	Date of payment

Foreign Key: taxpayer_id → Taxpayer

7.6 Tax_Adjustment Table

Stores excess tax carried forward.

Field Name	Data Type	Description
adjustment_id	INT (PK)	Unique adjustment ID
taxpayer_id	INT (FK)	References Taxpayer
previous_year	VARCHAR(10)	Previous financial year
excess_amount	DECIMAL(10,2)	Excess tax amount
adjusted_in_year	VARCHAR(10)	Year in which adjusted

Foreign Key: taxpayer_id → Taxpayer

7.7 Tax_Payment Table

Stores final payable or refund status.

Field Name	Data Type	Description
payment_id	INT (PK)	Unique payment record
taxpayer_id	INT (FK)	References Taxpayer
financial_year	VARCHAR(10)	Financial year
total_tax	DECIMAL(10,2)	Calculated tax
total_advance_paid	DECIMAL(10,2)	Sum of advance tax
adjusted_amount	DECIMAL(10,2)	Adjustment applied
final_payable	DECIMAL(10,2)	Remaining tax payable
status	VARCHAR(20)	PAID / PENDING / REFUND

Foreign Key: taxpayer_id → Taxpayer

Entity Relationship Concept

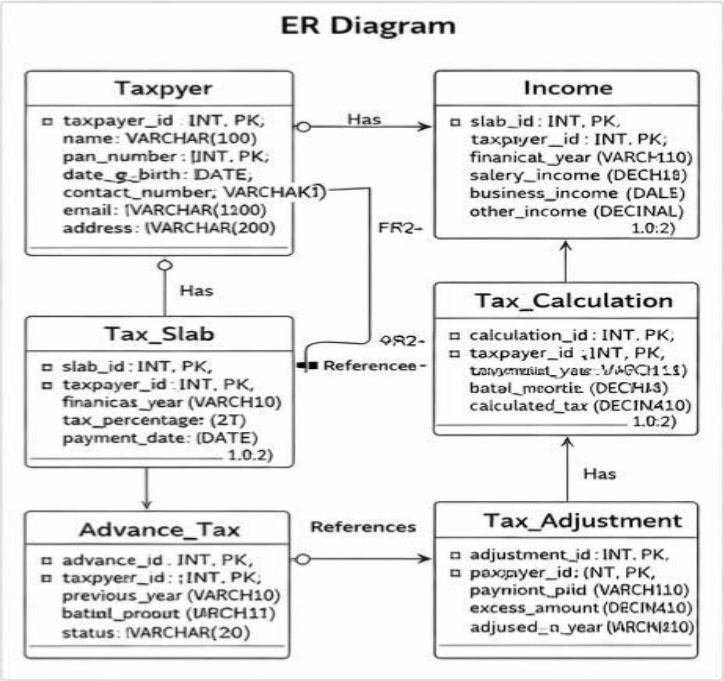
This relational structure ensures:

- Proper normalization
- Data integrity
- Support for advance tax logic
- Automatic carry-forward adjustment
- Financial year–wise tracking

8.UML DIAGRAMS

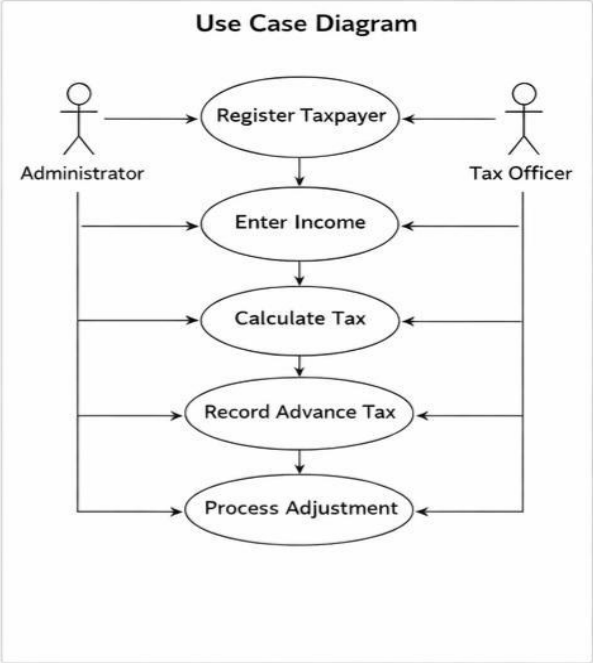
In this project, different UML diagrams are used to represent the structure and working of the Income Tax Record Management System with Advance Tax Method. These diagrams help to understand database structure, user interaction, workflow process, and system communication clearly.

8.1 ER Diagram



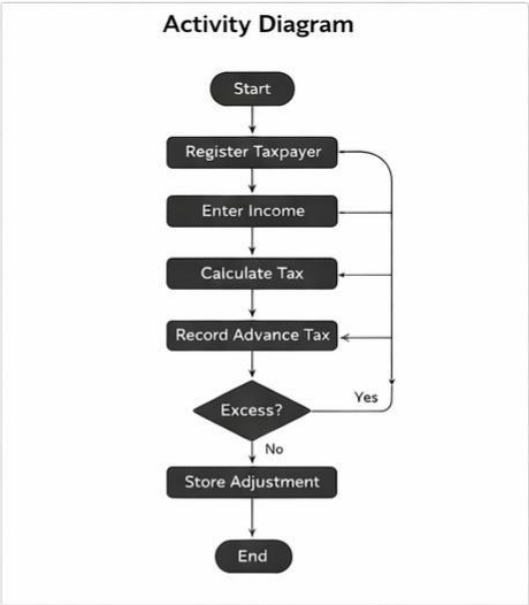
The ER Diagram represents the main entities of the system such as Taxpayer, Income, Tax_Slab, Advance_Tax, Tax_Calculation, and Tax_Adjustment. It shows how these entities are connected using primary and foreign keys. The diagram explains how income details are linked with tax calculation and how excess advance tax is adjusted and carried forward to the next financial year.

8.2 Use Case Diagram



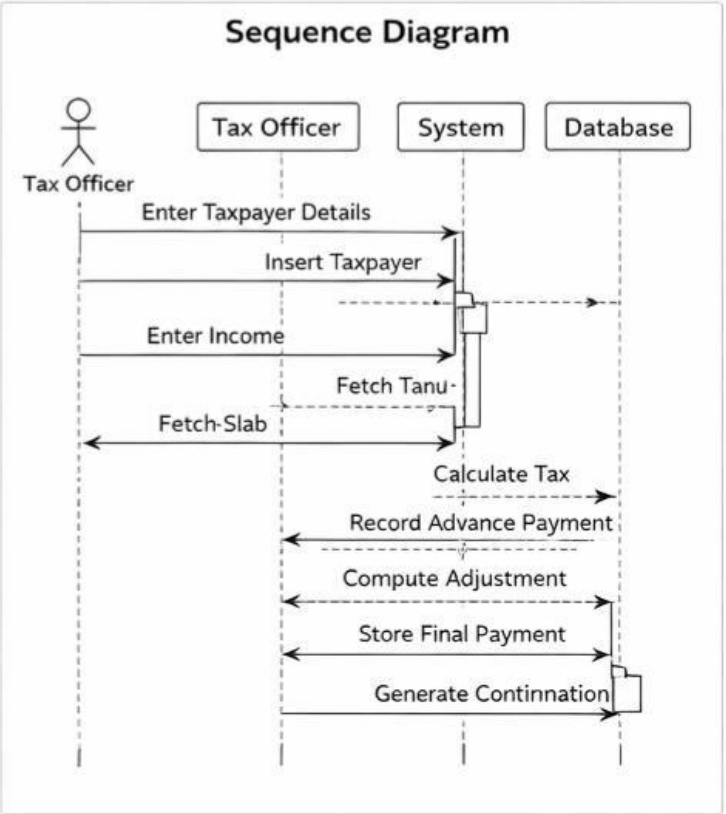
The Use Case Diagram shows the interaction between the Administrator and Tax Officer with the system. It includes operations like Register Taxpayer, Enter Income, Calculate Tax, Record Advance Tax, and Process Adjustment. This diagram explains how users perform tax-related operations and manage advance tax payments efficiently.

8.3 Activity Diagram



The Activity Diagram represents the step-by-step workflow of the system. It starts from taxpayer registration and income entry, then moves to tax calculation and advance tax recording. If excess tax is detected, the system processes adjustment and stores the updated tax details for the next year.

8.4 Sequence Diagram



The Sequence Diagram illustrates the interaction between the Tax Officer, System, and Database. It shows how taxpayer data is inserted, income details are processed, tax is calculated, advance payment is recorded, and adjustment is computed before storing the final payment information in the database.

9. SQL IMPLEMENTATION

- Database creation
- Table creation
- Advance tax logic
- Excess carry-forward logic
- Final payable calculation

9.1 Database Creation

```
CREATE DATABASE IncomeTaxDB;
USE IncomeTaxDB;
```

9.2 Table Creation

1.Taxpayer Table

```
CREATE TABLE Taxpayer (
    taxpayer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    pan_number VARCHAR(15) UNIQUE,
    date_of_birth DATE,
    contact_number VARCHAR(15),
    email VARCHAR(100),
    address VARCHAR(200)
);
```

2.Income Table

```
CREATE TABLE Income (
    income_id INT PRIMARY KEY AUTO_INCREMENT,
    taxpayer_id INT,
    financial_year VARCHAR(10),
    salary_income DECIMAL(10,2),
    business_income DECIMAL(10,2),
    other_income DECIMAL(10,2),
    total_income DECIMAL(10,2),
    FOREIGN KEY (taxpayer_id) REFERENCES Taxpayer(taxpayer_id)
);
```

3.Tax_Calculation Table

```
CREATE TABLE Tax_Calculation (
    calculation_id INT PRIMARY KEY AUTO_INCREMENT,
    taxpayer_id INT,
    financial_year VARCHAR(10),
    total_income DECIMAL(10,2),
    calculated_tax DECIMAL(10,2),
    FOREIGN KEY (taxpayer_id) REFERENCES Taxpayer(taxpayer_id)
);
```

4. Advance_Tax Table

```
CREATE TABLE Advance_Tax (
    advance_id INT PRIMARY KEY AUTO_INCREMENT,
    taxpayer_id INT,
    financial_year VARCHAR(10),
    installment_no INT,
    amount_paid DECIMAL(10,2),
    payment_date DATE,
    FOREIGN KEY (taxpayer_id) REFERENCES Taxpayer(taxpayer_id)
);
```

5. Tax_Adjustment Table

```
CREATE TABLE Tax_Adjustment (
    adjustment_id INT PRIMARY KEY AUTO_INCREMENT,
    taxpayer_id INT,
    previous_year VARCHAR(10),
    excess_amount DECIMAL(10,2),
    adjusted_in_year VARCHAR(10),
    FOREIGN KEY (taxpayer_id) REFERENCES Taxpayer(taxpayer_id)
);
```

6. Tax_Payment Table

```
CREATE TABLE Tax_Payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    taxpayer_id INT,
    financial_year VARCHAR(10),
    total_tax DECIMAL(10,2),
    total_advance_paid DECIMAL(10,2),
    adjusted_amount DECIMAL(10,2),
```

```
    final_payable DECIMAL(10,2),
    status VARCHAR(20),
    FOREIGN KEY (taxpayer_id) REFERENCES Taxpayer(taxpayer_id)
);
```

9.3 Insert Sample Data

1.Taxpayer

```
INSERT INTO Taxpayer
(name, pan_number, date_of_birth, contact_number, email, address)
VALUES
('Rahul Sharma','ABCDE1234F','1995-05-
10','9876543210','rahul@gmail.com','Mumbai');
```

2.Income

```
INSERT INTO Income
(taxpayer_id, financial_year, salary_income, business_income, other_income,
total_income)
VALUES
(1,'2025-26',800000,200000,50000,1050000);
```

9.4 Advance Tax Logic

```
INSERT INTO Tax_Calculation (taxpayer_id, financial_year, total_income,
calculated_tax)
SELECT
taxpayer_id,
financial_year,
total_income,
CASE
WHEN total_income <= 250000 THEN 0
WHEN total_income BETWEEN 250001 AND 500000
THEN (total_income - 250000) * 0.05
WHEN total_income BETWEEN 500001 AND 1000000
THEN (250000 * 0.05) +
     (total_income - 500000) * 0.20
WHEN total_income > 1000000
THEN (250000 * 0.05) +
     (500000 * 0.20) +
     (total_income - 1000000) * 0.30
END
From Income;
```

9.5 Advance Tax

```
INSERT INTO Advance_Tax
(taxpayer_id, financial_year, installment_no, amount_paid, payment_date)

VALUES
(1,'2025-26',1,150000,'2025-06-15');
```

9.6 Insert Final Tax Payment

```
INSERT INTO Tax_Payment
(taxpayer_id, financial_year, total_tax, total_advance_paid,
adjusted_amount, final_payable, status)

SELECT
tc.taxpayer_id,
tc.financial_year,
MAX(tc.calculated_tax),
IFNULL(SUM(at.amount_paid),0),
0,
(MAX(tc.calculated_tax) - IFNULL(SUM(at.amount_paid),0)),

CASE
WHEN (MAX(tc.calculated_tax) - IFNULL(SUM(at.amount_paid),0)) > 0 THEN
'PENDING'
WHEN (MAX(tc.calculated_tax) - IFNULL(SUM(at.amount_paid),0)) < 0 THEN
'REFUND'
ELSE 'PAID'
END

FROM Tax_Calculation tc
LEFT JOIN Advance_Tax at
ON tc.taxpayer_id = at.taxpayer_id
AND tc.financial_year = at.financial_year

GROUP BY tc.taxpayer_id, tc.financial_year;
```

9.7 Carry Forward Excess Tax to Next Year

```
INSERT INTO Tax_Adjustment
(taxpayer_id, previous_year, excess_amount, adjusted_in_year)

SELECT
taxpayer_id,
financial_year,
ABS(final_payable),
'2026-27'

FROM Tax_Payment
WHERE final_payable < 0;
```

9.8 Output Check

```
SELECT * FROM Taxpayer;
SELECT * FROM Income;
SELECT * FROM Tax_Calculation;
SELECT * FROM Advance_Tax;
SELECT * FROM Tax_Payment;
SELECT * FROM Tax_Adjustment;
```

This ensures:

- Advance tax comparison
- Automatic refund detection
- Excess carry-forward to next year
- Final payable status generation

10. SYSTEM TESTING AND RESULT

System testing was carried out to ensure that the Income Tax Record Management Database System functions correctly and produces accurate tax computation results using the advanced tax adjustment logic. Different SQL queries were executed in MySQL Workbench to verify the correctness of operations, data validation, tax calculation, advance tax payment recording, and final tax adjustment.

10.1 Query Correctness Testing

All SQL queries such as CREATE, INSERT, SELECT and transaction-related queries were tested individually to verify that:

- Tables were created successfully without errors
- Data was inserted accurately into the database
- Relationships between tables using foreign keys were maintained
- Advance tax payment entries were stored correctly
- Progressive tax calculation logic worked properly
- Excess advance tax was adjusted to the next financial year automatically

The output obtained confirmed that the database schema was properly structured and functional.

Database Table Verification

The following SQL query was executed to verify that all required tables were successfully created in the Income Tax database system.

SHOW TABLES

```
mysql> show tables;
+------------------------+
| Tables_in_advancetaxdb |
+------------------------+
| advance_tax            |
| income                 |
| tax_adjustment         |
| tax_calculation        |
| tax_payment            |
| taxpayer               |
+------------------------+
6 rows in set (0.04 sec)
```

This output confirms that all the necessary tables such as Taxpayer, Income, Tax_Calculation, Advance_Tax, Tax_Payment, and Tax_Adjustment were created successfully in the database.

10.2 Data Validation

Sample taxpayer information and income details were inserted into the respective tables to test data accuracy and integrity.

SELECT * FROM Taxpayer;

SELECT * FROM Income;

```
mysql> SELECT * FROM Taxpayer;
+-------------+--------------+------------+--------------+----------------+-----------------+---------+
| taxpayer_id | name         | pan_number | date_of_birth | contact_number | email           | address |
+-------------+--------------+------------+--------------+----------------+-----------------+---------+
|           1 | Rahul Sharma | ABCDE1234F | 1995-05-10    | 9876543210     | rahul@gmail.com | Mumbai  |
+-------------+--------------+------------+--------------+----------------+-----------------+---------+
1 row in set (0.11 sec)

mysql> SELECT * FROM Income;
+-----------+-------------+----------------+---------------+-----------------+--------------+--------------+
| income_id | taxpayer_id | financial_year | salary_income | business_income | other_income | total_income |
+-----------+-------------+----------------+---------------+-----------------+--------------+--------------+
|         1 |           1 | 2025-26        |     800000.00 |       200000.00 |     50000.00 |   1050000.00 |
+-----------+-------------+----------------+---------------+-----------------+--------------+--------------+
1 row in set (0.03 sec)
```

The output shows that taxpayer personal details and financial income information were stored accurately in the respective database tables.

10.3 Tax Calculation Verification

The system calculates the total tax liability based on the total annual income of the taxpayer using progressive tax logic implemented through SQL queries.

SELECT * FROM Tax_Calculation;

```
mysql> SELECT * FROM Tax_Calculation;
+----------------+-------------+----------------+--------------+----------------+
| calculation_id | taxpayer_id | financial_year | total_income | calculated_tax |
+----------------+-------------+----------------+--------------+----------------+
|              1 |           1 | 2025-26        |   1050000.00 |      127500.00 |
+----------------+-------------+----------------+--------------+----------------+
1 row in set (0.03 sec)
```

The result verifies that the calculated tax amount was correctly generated based on the taxpayer's total income for the financial year.

10.4 Advance Tax Payment Verification

Advance tax installment details paid by the taxpayer were recorded in the Advance_Tax table.

SELECT * FROM Advance_Tax;

```
mysql> SELECT * FROM Advance_Tax;
+------------+------------+----------------+----------------+-------------+--------------+
| advance_id | taxpayer_id | financial_year | installment_no | amount_paid | payment_date |
+------------+------------+----------------+----------------+-------------+--------------+
|          1 |          1 | 2025-26        |              1 |   150000.00 | 2025-06-15   |
+------------+------------+----------------+----------------+-------------+--------------+
1 row in set (0.04 sec)
```

The output confirms that advance tax payments made during the financial year were successfully recorded in the system.

10.5 Final Tax Payment and Adjustment

The system automatically compares the calculated tax with the total advance tax paid to determine the final payable tax or refund status.

SELECT * FROM Tax_Payment

```
mysql> SELECT * FROM Tax_Payment;
+------------+------------+----------------+-----------+--------------------+-----------------+---------------+--------+
| payment_id | taxpayer_id | financial_year | total_tax | total_advance_paid | adjusted_amount | final_payable | status |
+------------+------------+----------------+-----------+--------------------+-----------------+---------------+--------+
|          1 |          1 | 2025-26        | 127500.00 |          150000.00 |            0.00 |     -22500.00 | REFUND |
+------------+------------+----------------+-----------+--------------------+-----------------+---------------+--------+
1 row in set (0.00 sec)
```

The result indicates that the advance tax paid was greater than the calculated tax liability, resulting in a refund status.

10.6 Carry Forward of Excess Advance Tax

If the advance tax paid exceeds the actual tax liability, the system automatically carries forward the excess amount to the next financial year using the Tax Adjustment module.

SELECT * FROM Tax_Adjustment;

```
mysql> SELECT * FROM Tax_Adjustment;
+---------------+------------+---------------+---------------+------------------+
| adjustment_id | taxpayer_id | previous_year | excess_amount | adjusted_in_year |
+---------------+------------+---------------+---------------+------------------+
|             1 |          1 | 2025-26       |      22500.00 | 2026-27          |
+---------------+------------+---------------+---------------+------------------+
```

The output confirms that the excess tax amount was successfully recorded and adjusted for the next financial year.

10.7 Result Analysis

The results obtained from system testing show that the database system operates correctly and efficiently. All SQL queries executed without errors, taxpayer records were stored accurately, and the tax computation process was performed successfully. The system correctly identified excess advance tax payments and carried forward the refundable amount to the subsequent financial year. Overall, the outputs matched the expected results, proving that the implemented advanced tax adjustment logic meets the project requirements.

11. SECURITY, BACKUP AND RECOVERY

11.1 Security Measures

- Database authentication with username/password.
- Role-based access: Admin, Tax Officer, Consultant.
- Use of PRIMARY and FOREIGN KEYS to prevent unauthorized data changes.
- Sensitive data (PAN, income) stored securely with proper constraints.
- SQL injection prevention via parameterized queries (if using an application layer).

11.2 Backup Strategy

- Daily backup using `mysqldump`:

```
mysqldump -u root -p IncomeTaxDB > IncomeTaxDB_backup.sql
```

- Weekly full database backups.
- Backup files stored securely on external storage or cloud.

11.3 Recovery Procedure

- Restore from backup in case of data loss:

```
mysql -u root -p IncomeTaxDB < IncomeTaxDB_backup.sql
```

- Ensures minimal downtime and integrity of taxpayer records.

12.1 FUTURE SCOPE

The system can be further enhanced to include:

1. **Integration with Government Portals** – Direct syncing with e-filing and TDS portals.
2. **Web-based Interface** – GUI for users instead of SQL Workbench.
3. **Mobile Application Support** – Taxpayers can check status and pay online.
4. **Automated Notifications** – Email/SMS reminders for advance tax installments.
5. **Advanced Tax Rules** – Include exemptions, rebates, and deductions.
6. **Data Analytics & Reports** – Visual dashboards for tax trends and summaries.
7. **Multi-year Reporting** – Compare past financial years for better planning.
8. **Enhanced Security** – Encryption of sensitive fields and audit logs.

12.2 CONCLUSION

The Income Tax Record Management System with Advance Tax Adjustment demonstrates how a relational database can efficiently handle:

- Structured storage of taxpayer data
- Accurate slab-based tax computation
- Tracking of advance tax installments
- Automatic detection and carry-forward of excess tax
- Final payable and refund calculation
- Transaction management for safe updates

This system reduces manual errors, improves reliability, and provides practical understanding of real-world taxation integrated with database management concepts. It serves both academic and small-scale consultancy purposes.

13. REFERENCES

1. **MySQL Documentation** – https://dev.mysql.com/doc/
2. Elmasri, R., & Navathe, S. **Fundamentals of Database Systems**, 7th Edition.
3. Income Tax Department of India – https://www.incometaxindia.gov.in
4. Korth, H., & Silberschatz, A. **Database System Concepts**, 7th Edition.
5. Practical examples and tutorials from **W3Schools SQL** – https://www.w3schools.com/sql/

14. GLOSSARY

- **Advance Tax:** Tax paid in installments before the final assessment.
- **TDS (Tax Deducted at Source):** Tax deducted by employer or payer before payment.
- **Carry-Forward Adjustment:** Excess tax from a financial year applied to the next year's liability.
- **SQL (Structured Query Language):** Language used to manage relational databases.
- **PRIMARY KEY (PK):** Unique identifier for a table row.
- **FOREIGN KEY (FK):** Field linking two tables for referential integrity.
- **Transaction:** A sequence of database operations executed as a single unit (COMMIT/ROLLBACK).
- **Refund:** Excess tax returned to the taxpayer.
- **PENDING:** Tax liability remaining to be paid.
- **PAID:** Tax liability fully paid.